
MULTIDIMENSIONAL PARAMETER DECOMPOSITION FOR MECHANISTIC INTERPRETABILITY

A PREPRINT

Knud Rasmus Sønderbek

May 13, 2026

ABSTRACT

Some super cool abstract of all my findings ¹

1 Introduction

Mechanistic interpretability seeks to reverse-engineer neural networks into a set of human-interpretable mechanisms. Different branches of the field pursue this goal with different focal points: directions in activation space, computational subgraphs, or components of the network’s parameters.

However, both the atomic structure for decomposition and the decomposition itself remain open problems. [Sha+25] Whether a decomposition recovers the computational primitives of a neural network depends on what the ground-truth decomposition is taken to be. With models consisting of independent features, the decomposition is unambiguous, with one atom per feature. If the features are dependent or co-occur, the decomposition becomes underdetermined. Decompositions that split features across multiple atoms, risk losing the structural information, that gives each feature its meaning, while decompositions that group multiple features into a single atom, risk losing granularity by collapsing distinct features into a single unit.

For example, a network computing the volume of a cube can be decomposed into components corresponding to individual side lengths, or into a single component computing the volume directly. The former corresponds to a feature-level decomposition, with one atom per input direction, exposing internal structure but leaving grouping implicit, only recoverable through co-activation patterns. The latter corresponds to a structural-level decomposition, with one atom per computation, exposing the grouping directly but collapsing per side-length information.

Recently, a formal definition of irreducibility was introduced to disambiguate cases such as the cube example. [Eng+25] Features are irreducible, and thus constitute units of decomposition, if their joint distribution cannot be decomposed into independent factors or into a mixture containing a lower-dimensional component.

In terms of the decomposition itself, recent work focuses on decomposing parameters, as opposed the activations. These methods aim to recover components that are faithful to the original weights, sparse per input, and individually simple. One such approach, Stochastic Parameter Decomposition (SPD) [BBS25], factorizes networks into rank-one components with a learned per component importance function.

This work extends SPD by examining its behaviour on dependent features, in settings where the multi-dimensional feature hypothesis posits a feature-level or a structural-level unit ² [Eng+25]. Additionally a reformulation of the importance function is proposed, to encourage that features that are manipulated jointly are grouped together, by computing importance relationally across components.

My experiment findings

¹Very important to add quantization somewhere. <https://arxiv.org/pdf/2303.13506>

²I mean feature-level, structure level is wrong under this view but it sounds good

2 Related work

2.1 The linear representation hypothesis

A widespread view in mechanistic interpretability is the linear representation hypothesis: high-level concepts are encoded as directions in a network’s representation space [PCV24]. Together with the superposition hypothesis [Elh+22], it forms a central tenet of interpretability: each feature corresponds to a single direction, and a hidden state is a sparse linear combination of feature directions.

Recent work questions how strictly this view should be taken, distinguishing between a weak form of the hypothesis, that some features are linear, from a stronger form that all features are [Men24; Smi24]. Empirically multiple counterexamples to the stronger form have been proposed, include recurrent networks encoding token position by magnitude rather than direction [Cso+24], transformers representing cyclical concepts such as days of the week [Eng+25] or modular arithmetic [Nan+23] in irreducible low-dimensional subspaces.

2.2 Activation-space decomposition

Decomposing a network’s activations predominantly focuses on disentangling layer activations into interpretable features. Here, sparse autoencoders (SAE) are one of the most widely researched methods [Cun+23; Bri+23], that learn an overcomplete dictionary that expresses each hidden state as a sparse combination of one-dimensional features. However, these features are not necessarily aligned with the network’s computation, due to feature absorption, inconsistencies across dictionary sizes, and a failure to capture feature geometry. [Cha+25; Lea+25; Men24].

Circuit discovery extends feature analysis to identify how a computation is processed by the network’s structure, by identifying minimal subgraphs required to perform a given task, without loss of performance. [Ola+20; Mar+25]. However, these methods generally do not account for the polysemantic and densely connected structure of neural networks, where the functional behaviour can be attributed to multiple distinct but functionally equivalent subgraphs [Mél+25].

2.3 Parameter-space decomposition

Recently a newer line of work aims to decompose the network’s parameters directly, instead of the activations. Transcoders [DCN24] represent an intermediate step that learns a sparse approximation of an MLP layer. Parameter decomposition methods go further, factorising the network’s weights directly, with the goal of recovering the network’s computational primitives.

Most directly relevant to this work is the description-length-driven line of Attribution-based Parameter Decomposition [Bra+25] and Stochastic Parameter Decomposition (SPD) [BBS25], which has recently been explored in small transformer models [CR25]. While APD decomposes a network into multidimensional components, the method is sensitive to hyperparameters, and does not scale beyond toy settings due to computational complexity. SPD addresses this by decomposing each weight matrix into rank-one subcomponents and learning a per-input importance function over them, at the cost of requiring post-hoc clustering to recover higher-rank mechanisms; both methods are detailed in Section 3.1. Other parameter-space approaches include local loss landscape decomposition [CBS25].

3 Method

3.1 Background on SPD

Attribution-based Parameter Decomposition (APD) [Bra+25] and Stochastic Parameter Decomposition (SPD) [BBS25] both aim to decompose a network’s parameters into a set of components that are faithful to the original weights, minimal in the number required per input, and individually as simple as possible.

To formalise this, consider a neural network $f(x, W)$ with weight matrices $W = \{W^1, \dots, W^L\}$. A set of parameter components $\{W_c^l\}_{c=1}^C$ are said to comprise the network’s mechanisms if they satisfy the following properties:

- **Faithfulness:** The parameter components should sum to the parameters of the original network.
- **Minimality:** As few parameter components as possible should be required to replicate the network’s output on any given input.
- **Simplicity:** Each component should span as few weight matrices and as few ranks as possible.

APD [Bra+25] approaches this by decomposing the network into a set of parameter components without constraining their rank, using a gradient-based attribution that selects the top- k components most responsible for any given input. While effective on small models, APD scales poorly: its memory cost grows with the number of components, as each component has the same dimensionality as the original network, and its top- k attribution is sensitive to hyperparameter choice.

SPD [BBS25] addresses these limitations by enforcing a rank-one structure on each component, allowing more components to be learned at fixed memory cost, and replacing top- k attribution with a learned causal importance function. Formally, each weight matrix W^l is decomposed into a sum of C rank-one subcomponents:

$$W_{i,j}^l \approx \sum_{c=1}^C U_{c,i}^l V_{c,j}^l, \quad \text{where } U^l \in \mathbb{R}^{C \times d_{\text{out}}}, V^l \in \mathbb{R}^{C \times d_{\text{in}}} \quad (1)$$

where C is a hyperparameter, for the number of components for each layer, l indexes the layer and i, j are the hidden indices. The subcomponent count, C , may be chosen to be larger than both d_{in} and d_{out} , enabling the method to learn features in superposition [BBS25].

To enforce the properties above, three losses are introduced, which are optimised jointly. Faithfulness is enforced through a mean squared error loss between the original network’s weights and the the sum of subcomponents:

$$\mathcal{L}_{\text{faithfulness}} = \frac{1}{N} \sum_{l=1}^L \sum_{i,j} \left(W_{i,j}^l - \sum_{c=1}^C U_{i,c}^l V_{c,j}^l \right)^2 \quad (2)$$

where N is the total parameter count in the target model.

Optimization for simplicity and minimality is achieved through a stochastic masking procedure. Intuitively, if a subcomponent is not necessary for reconstruction, it can be sampled to zero without loss of faithfulness. A causal importance function $\Gamma : \mathcal{X} \rightarrow [0, 1]^{C \times L}$ is trained to learn the lower bound of the sampling distribution. In SPD, Γ is a set of independent MLPs γ_c^l that each take a layer’s activation and output a scalar importance for the corresponding subcomponent:

$$g_c^l(x) = \Gamma_c^l(x) = \sigma_H(\gamma_c^l(h_c^l(x))), \quad h_c^l(x) = \vec{V}_c^{l\top} \vec{a}^l(x) \quad (3)$$

where σ_H is a hard sigmoid and $\vec{a}^l(x)$ is the activation at layer l . These importance values serve as the lower bound for how much a subcomponent can be masked. With $r_c^l \sim \mathcal{U}(0, 1)$ sampled per subcomponent, the masked weight matrix is given by:

$$\begin{aligned} m_c^l(x, r) &= g_c^l(x) + (1 - g_c^l(x)) r_c^l, \\ W^l(x, r) &= \sum_{c=1}^C \vec{U}_{i,c}^l m_c^l(x, r) \vec{V}_{c,j}^l, \end{aligned} \quad (4)$$

These masked matrices are trained to reconstruct the original network’s output.

$$f(x \mid \{W^l(x, r)\}_{l=1}^L) \approx f(x \mid \{W^l\}_{l=1}^L). \quad (5)$$

This is done through both a layer-wise and a global reconstruction loss. The layer-wise version is necessary to provide a stronger training signal as randomly masking all components at the same time can be too noisy to learn from.

$$\begin{aligned} \mathcal{L}_{\text{stochastic-recon}} &= \frac{1}{S} \sum_{s=1}^S D(f(x \mid W^l(x, r^s)), f(x \mid W)), \\ \mathcal{L}_{\text{stochastic-recon-layer}} &= \frac{1}{LS} \sum_{l=1}^L \sum_{s=1}^S D(f(x \mid W^1, \dots, W^l(x, r^{(s)}), \dots, W^L), f(x \mid W)). \end{aligned} \quad (6)$$

where D is a task-specific divergence measure, such as MSE or KL divergence.

Lastly, to enforce minimality, the importance function is trained to only active subcomponents that are necessary for reconstruction. Thus becoming a trade-off between faithfulness and minimality, where the importance function learns to only activate the subcomponents that are genuinely required to reconstruct the target model’s output.

$$\mathcal{L}_{\text{minimality}} = \sum_{l=1}^L \sum_{c=1}^C (g_c^l(x))^p, \quad p > 0. \quad (7)$$

The full SPD objective is a weighted sum of these terms:

$$\mathcal{L}_{\text{SPD}} = \mathcal{L}_{\text{faithfulness}} + \beta_1 \mathcal{L}_{\text{stochastic-recon}} + \beta_2 \mathcal{L}_{\text{stochastic-recon-layer}} + \beta_3 \mathcal{L}_{\text{minimality}}. \quad (8)$$

3.2 Irreducibility and the natural unit of decomposition

To appropriately evaluate the performance of a multi-dimensional feature decomposition method, it's necessary to have a clear definition of what the ground-truth decomposition is. The original superposition hypothesis states that the hidden states can be represented as a linear combination of sparse one-dimensional features. [Elh+22; Eng+25]

Definition: The hidden state $x_{i,l}$ can be written as:

$$x_{i,l} = \sum_i v_i f_i(t) \quad (9)$$

where $v_i \in \mathbb{R}^d$ are pairwise δ -orthogonal directions ³ and $f_i(t) \in \mathbb{R}$ are sparse scalar activations, with $f_i(t) = 0$ outside the domain of feature i .

Recent work has shown, that this may be insufficiently strict [Smi24; Eng+25], as it does not capture the structure of multi-dimensional features. In particular, transformer models are shown to organise cyclical concepts in low-dimensional subspaces that do not decompose into meaningful linear components, but are instead manipulated jointly [Eng+25].

To accomodate these features, the same paper proposes a correction to the superposition hypothesis, alongside a formal definition of irreducibility that defines, when a feature is in its atomic form and when it can be decomposed into smaller units without loss of information.

Definition: A feature $f \in \mathbb{R}^{d_f}$ is truly multi-dimensional, if it cannot be written as a sum of two independent features or as the sum of two non-co-occurring features. More formally, a feature is reducible into features a and b if there exists an affine transformation

$$f \mapsto Rf + c \equiv \begin{pmatrix} a \\ b \end{pmatrix} \quad (10)$$

for some orthonormal $d_f \times d_f$ matrix R , and additive constant c , such that the joint distribution $p(a, b)$ satisfies one of two conditions:

- **p is separable:** $p(a, b) = p(a) p(b)$
- **p is a mixture of two disjoint distributions:** $p(a, b) = w p(a) \delta(b) + (1 - w) p'(a, b|b \neq 0)$,

Here a and b are the first k and remaining $d_f - k$ components of $Rf + c$, p is conditioned on inputs where f is active, $w \in (0, 1)$ and δ is the Dirac delta ⁴. In the mixture, the two distributions have disjoint support, with one distribution having no mass where b is active, and the other having no mass where b is zero.

Under this decomposition, the superposition hypothesis takes the form:

Definition: Every hidden state $x_{i,l}(t)$ can be written as a sum of feature contributions

$$x_{i,l}(t) = \sum_i V_i f_i(t) \quad (11)$$

where each $f_i(t) \in \mathbb{R}^{d_{f_i}}$ is an irreducible feature with internal dimensionality $d_{f_i} \geq 1$, each $V_i \in \mathbb{R}^{d \times d_{f_i}}$ and $\|V_i^\top V_j\| < \delta^5$ for $i \neq j$.

The definition of $V_i \in \mathbb{R}^{d \times d_{f_i}}$ does not require its columns to be linearly independent: d_{f_i} specifies an upper bound on the representational capacity of V_i , while $\text{rank}(V_i) \leq d_{f_i}$ determines the effective intrinsic dimensionality required to represent the feature. In practice, this allows models to align dependent components within a shared lower-dimensional subspace (Section ??). Consequently, features need not correspond to rank-one directions, but may instead occupy structured higher dimensional spaces. Decompositions restricted to rank-one components are therefore not, in general, aligned with this representation without post-hoc clustering.

³should i say what delta is or is it clear that its small?

⁴can i get away with just writing $p(a|b = 0)$

⁵again do i need to define delta?

3.3 Rank- k subcomponents and relational importance

In the original SPD paper, the subcomponents are constructed as rank-one matrices, through an outer product of two vectors. This formulation is however incompatible with the multi-dimensional superposition hypothesis. (Section 3.2) A decomposition into lower-dimensional subcomponents, necessarily forces features to be split across multiple subcomponents, or to be approximated through a single subcomponent at the cost of reconstruction.

To represent multi-dimensional features, the pairs of vectors are replaced with pairs of matrices.

$$W_{i,j}^l \approx \sum_{c=1}^C \sum_{r=1}^k U_{c,i,r}^l V_{c,j,r}^l, \quad \text{where } U^l \in \mathbb{R}^{C \times d_{\text{out}} \times k}, V^l \in \mathbb{R}^{C \times d_{\text{in}} \times k} \quad (12)$$

Each subcomponent $U_c^l V_c^{l\top} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ has rank at most k , recovering the original SPD definition when $k = 1$. As in the multi-dimensional superposition hypothesis, k is an upper bound on each subcomponent’s representational capacity; the effective rank may be lower, as it will be shown in Section 3.3.1.

In SPD, each subcomponent’s importance is computed independently through an MLP followed by a sigmoid non-linearity, allowing multiple subcomponents to be active for a given input and thus distribute a feature across them, even when it is irreducible. Likewise, the minimality loss does not translate to the rank- k setting, where each subcomponent can represent multiple feature directions: it imposes no penalty for distributing a feature across multiple subcomponents, nor for a single subcomponent to encode parts of multiple features. (Section 3.1)

To address this, the causal importance function is redefined to maximize the conditional probability, that a given subcomponent W_c^l , is responsible for a given feature, conditioned on the activations of all other subcomponents in the same layer.

$$g_c^l(x) = p(W_c^l \mid \{h_{c'}^l(x)\}_{c'=1}^C), \quad h_c^l(x) = V_c^{l\top} \bar{a}^l(x) \in \mathbb{R}^k \quad (13)$$

where $\bar{a}^l(x)$ is the activation of layer l and C is the set of subcomponents in layer l . The distribution is computed with a classic transformer block operating over subcomponents, with a softmax over their activations. [Vas+23]

This produces a normalized assignment over subcomponents, $\sum_{c=1}^C g_c^l(x) = 1$, causing the original minimality loss to become degenerate, as each layer contributes a constant value independent of the model state. Instead, it’s replaced with entropy, to pressure the assignment to concentrate on as few subcomponents as possible.

$$\mathcal{L}_{\text{minimality}} = -\frac{1}{|L|} \sum_{l=1}^L \sum_{c=1}^C g_c^l(x) \log g_c^l(x) \quad (14)$$

⁶ This pressure is complementary to both the faithfulness loss and the rank- k extension. If k is too small or the minimality coefficient is too small, the faithfulness loss dominates and forces features to split across subcomponents. Conversely, if the minimality weight is too high, the model groups features together at the cost of faithfulness.

An unfortunate limitation of this formulation is its computational cost. Computing relational importance via a transformer over subcomponents scales quadratically with the number of subcomponents compared to the linear cost of the original independent importance functions. ⁷

3.3.1 SVD reformulation, capacity maybe per feature ablation TBD

+ add read write streams, theyre a general thing -i perhaps issues with diffusion and recurrent models where this stream is not as clear. <https://arxiv.org/pdf/2504.00194> + This section must explain delta orthogonality, visualization of Read, write matrices, perhaps nudge the SVD view. Most importantly should address if models learn lower dimensional representations or if its another k sensitivity as in APD.

The competition pressure introduced by the softmax has a theoretical motivation under the multi-dimensional superposition hypothesis. The hypothesis predicts that different irreducible features occupy approximately orthogonal projection subspaces; the softmax-and-attention mechanism implements a training pressure toward this orthogonality at the level of SPD’s atoms. Atoms whose V matrices share input directions cannot be cleanly distinguished on inputs that activate those directions, and the softmax penalises ambiguous allocations. The optimiser’s response is to push the atoms’ V matrices toward disjoint subspaces, recovering the orthogonality structure the hypothesis posits.

Remember to add an experiment that shows if softmax enforce the orthogonality constraint from engels.

⁶Not sure if C or c times L

⁷Play with attention approximations

4 Results

4.1 Overview of experiments

To evaluate how the proposed changes affect SPD’s performance, and how these relate to the multi-dimensional superposition hypothesis, the result section is structured around four experiments, each with the purpose of isolating a specific structural property. For an overview see table 1.

The first experiment reuses the original SPD toy models: Toy Model of Superposition (TMS), and Toy Model of compressed computation [BBS25]. These models consist of independent rank 1 features, and thus serves as a baseline that shows that softmax performs on par with the original sigmoid gating mechanism. Additionally it compares the sensitivity to the minimality hyperparameter.

The second experiment introduces a minimal extension: features from the Toy Model of Superposition are sampled in pairs, creating statistically dependent multi-dimensional features. Here, the Toy model learns to group the paired features along a single direction, allowing rank 1 subcomponents to capture full features, without having to split them across multiple subcomponents.

The third experiment generalizes the paired setting to a k-dimensional case by computing the volume of a simplex from its vertices. While the ground-truth decomposition remains simple, each feature is now k-dimensional, showing how the decomposition changes across different subcomponent rank constraints.

Finally, the fourth experiment trains a model to learn a logical expression composed of two disjoint distributions, one of which is lower-dimensional. This experiment tests the final condition of the irreducibility definition (Section 3.2).

	SPD Baseline	Toy Model of Superposition with tied weights	Toy Model of Geometry	Toy Model of logical superposition
Objective	$x \mapsto x$	$x \mapsto x$	$v \mapsto \text{vol}(v)$	$x \mapsto a \wedge b$
Superposition	✓	✓	×	✓
Irreducible	Trivially	✓	✓	×
Disjoint mixture	×	×	×	✓
Statistical dependence	×	✓	✓	✓
Feature dim / Model rank	1 / 1	2 / 1	$k / \leq k$	$k / \leq k$

Table 1: Overview of the toy models and their properties. Each experiment is chosen to isolate a specific property of the irreducibility definition. The baseline is trivially irreducible: a 1D scalar cannot be split into multiple components.

4.2 Toy Model of Superposition

The Toy Model of Superposition (TMS) [Elh+22] is trained to reconstruct sparse feature activations through a lower-dimensional bottleneck that forces the model to learn features in superposition. The architecture is a single-hidden-layer autoencoder with tied weights:

$$\hat{x} = \text{ReLU}(W^\top Wx + b), \quad W \in \mathbb{R}^{d_{\text{hidden}} \times d_{\text{features}}}, \quad d_{\text{features}} \gg d_{\text{hidden}}. \quad (15)$$

TMS is studied in two regimes: one with independent features, reproducing the original SPD setup, and one with paired features that introduces statistical dependence. For both regimes, the rank of each subcomponent is kept at 1, both because this is the ground-truth structure and to isolate the effect of softmax-and-attention.

To quantify how well the learned subcomponents recover the target model, two metrics are computed across both regimes. The mean max cosine similarity (MMCS)⁸ measures directional alignment between each target feature and its best-matching learned subcomponent:

$$\text{MMCS} = \frac{1}{d_{\text{features}}} \sum_{j=1}^{d_{\text{features}}} \max_c \left(\frac{\vec{U}_{:,c} V_{c,j} \cdot W_{:,j}}{\|\vec{U}_{:,c} V_{c,j}\|_2 \|W_{:,j}\|_2} \right). \quad (16)$$

⁸Remember to add reference

where $c \in C$ is the parameter component index and $j \in \{1, \dots, d_{\text{features}}\}$ is the input feature index. A value of 1 indicates that every target feature has a perfectly aligned subcomponent.

Apart from directional alignment, the strength of the alignment is also measured, to detect feature shrinkage, a problem apparent in both Sparse Autoencoders and Attribution-Based Parameter Decomposition [JT24; WS24; Bra+25]. The mean L_2 ratio (ML2R) compares the euclidean norm of the best-aligned subcomponent to the target feature:

$$\text{ML2R} = \frac{1}{d_{\text{features}}} \sum_{j=1}^{d_{\text{features}}} \frac{\|\vec{U}_{:, \text{mcs}(j)} \vec{V}_{\text{mcs}(j), :}\|_2}{\|W_{:, j}\|_2} \quad (17)$$

where $\text{mcs}(j)$ is the index of the most aligned subcomponent for feature j . A ratio near 1 indicates the feature is fully captured by its best-aligned subcomponent.

4.2.1 Independent features

Each feature is drawn independently from a uniform distribution over $[0, 1]$. As each feature is one-dimensional and independent of the others, the ground-truth decomposition is a set of d_{features} rank-one matrices, each non-zero only along the c -th column $W_{:, c}$ of the target weights.

Like in the original SPD paper, two TMS models are trained: TMS₅₋₂ with $d_{\text{features}} = 5$ and $d_{\text{hidden}} = 2$, and TMS₄₀₋₁₀ with $d_{\text{features}} = 40$ and $d_{\text{hidden}} = 10$ with an additional identity layer.

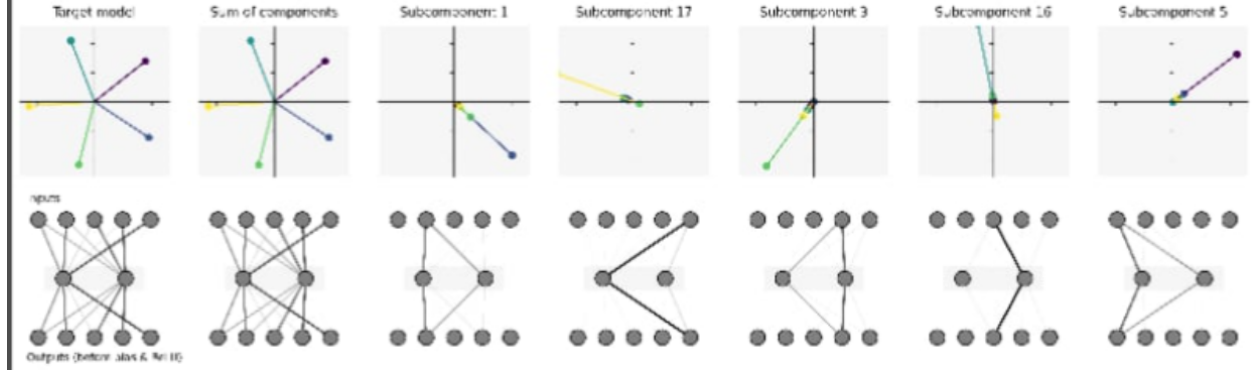


Figure 1: Decomposition of TMS₅₋₂. The first two panels show the target model’s weight columns and the sum of learned subcomponents. Subsequent panels show the individual subcomponents, ordered by decreasing norm. Although 20 subcomponents are allocated, only the first 5 carry meaningful norm and contribute to reconstruction, while the remaining 15 have negligible weight and are effectively pruned by the method. Each active subcomponent recovers primarily one feature direction, with minor leakage across components.

SPD achieves near perfect alignment on both models, with MMCS and ML2R approximately equal to 1. The softmax and attention variant, while achieving near-zero losses, does not manage to perfectly align its components with the target model nor concentrate the entire feature magnitude within a single subcomponent, resulting in a lower ML2R with high variance. For the larger model this skews the geometry further.

While the softmax variant does not recover the exact geometry, it correctly concentrates the entire magnitude on the right number of subcomponents, with the rest pruned to near-zero (Figure 1). In terms of causal importance values, the softmax and original SPD method show the same one-hot assignment for each of the five features in TMS₅₋₂, with some degradation in the larger model where certain subcomponents are causally important for multiple features.

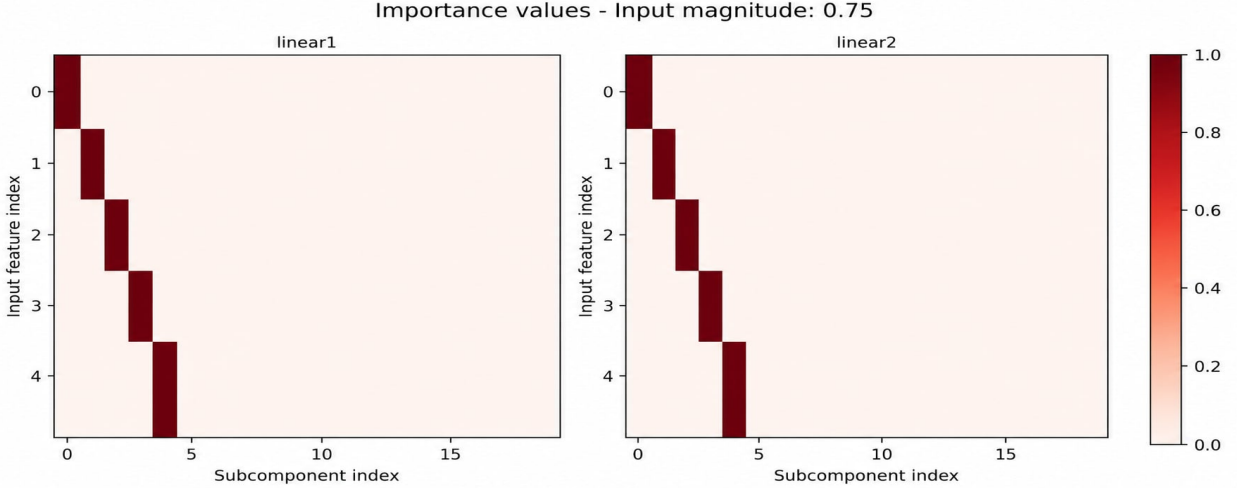


Figure 2: Causal importance values for the softmax variant on TMS_{5-2} , showing a one-hot assignment for each of the five features.

4.2.2 Paired features

In this regime, features are sampled in pairs to create the simplest form of dependence. Features are partitioned into pairs $\{f_{2i-1}, f_{2i}\}_{i=1}^{20}$, with each pair zero with probability $1-p$, and otherwise each pair member drawn independently from $\mathcal{U}(\epsilon, 1)$. The setup uses $d_{\text{features}} = 40$ and $d_{\text{hidden}} = 10$.

Each paired feature is a 2-dimensional input feature, but the trained model aligns both pair members along a single direction (Table 2), with within-group cosine similarity averaging 0.999 and across-group similarity averaging 0.053. The pair’s hidden representation therefore collapses to a 1D subspace. Due to this lower-dimensional mapping, paired features can be captured with a single rank-1 subcomponent, without needing to split them across multiple subcomponents.

Group	Mean	Std Dev	Range
Within-group	0.999	0.010	[0.997, 1.000]
Across-group	0.053	0.223	[-1.000, 0.038]

Table 2: Cosine similarity between encoder columns $W_{:,i}$ and $W_{:,j}$, partitioned by group membership.

The ground-truth decomposition is therefore 20 rank-1 subcomponents, with feature indices $\{2i-1, 2i\}$ assigned to subcomponent i , which the causal importance function should recover.

Both methods achieve near-perfect reconstruction and faithfulness, with losses around 10^{-6} , but differ significantly in their importance predictions. SPD correctly identifies that features should be paired in single subcomponents but does not isolate each pair into its own subcomponent, with several subcomponents carrying importance for multiple feature pairs. The softmax and attention variant correctly assigns each feature pair to a single subcomponent, with the exception of one pair (Figure 3).

	TMS_{5-2}		TMS_{40-10}		TMS_{40-10} paired	
	SPD	Softmax	SPD	Softmax	SPD	Softmax
MMCS	1.00	0.985 ± 0.016	1.00	0.812 ± 0.105	0.9997	0.9920 ± 0.003
ML2R	1.00	0.797 ± 0.205	1.055 ± 0.045	0.623 ± 0.231	0.796 ± 0.434	0.732 ± 0.350

Table 3: Decomposition quality across both TMS regimes. SPD refers to the original method [BBS25]; Softmax refers to the relational importance function of Section 3.3. If the standard deviation is not reported, it’s because its smaller than 0.001.

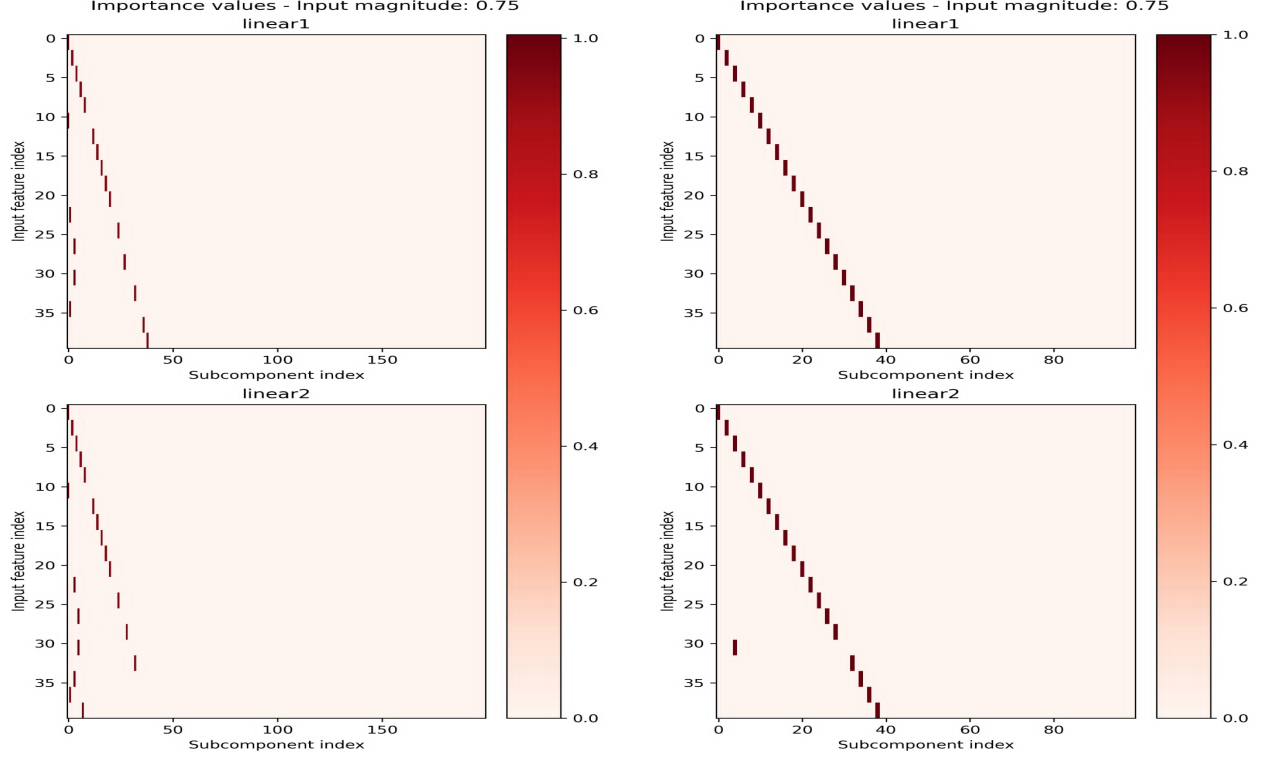


Figure 3: Left: SPD decomposition of the paired TMS model. Right: Softmax of the paired TMS model. SPD finds the correct structure of pairing features together but does not attribute one feature pair per subcomponent. Softmax achieves the desired importance decomposition, with the exception of one feature pair. SPD was trained with 200 subcomponents and Softmax with 100, as these yielded the best results for each method.

Overall across the three settings, SPD with sigmoid gating produces better results on both MMCS and ML2R, despite the more entangled importance assignments observed in the paired regime (Figure 3). The MMCS gap is generally small, but with substantially larger variation in ML2R. Notably, lower ML2R values are frequently associated with decompositions in which individual SPD subcomponents are assigned importance for multiple features.

Referring back to the TMS₅₋₂ decomposition shown in Figure 1, the softmax method appears to recover a slightly rotated geometry, with residual feature magnitude distributed across adjacent features. The geometric offset may arise as a compensatory mechanism for this residual leakage, allowing the combined subcomponents to maintain accurate reconstruction despite imperfect feature isolation.

Add the part about how We get one-hot importance values, but they don't seem to contribute to the input * output reading plot.

Could this be because of U? that is technically not penalized also comment on the norm.

I think this is the solution: After the first hidden layer, the projection lives in a 50 dimensional subspace, but we have an overcomplete basis with the 100 subcomponents, necessarily these cannot project into 100 orthogonal directions, so they must contribute to multiple features.

4.2.3 Toy model of compressed computation

The Toy Model of Compressed Computation [Bha+25; BBS25; Bra+25], is a toy model trained to learn the function $y_i = x_i + \text{relu}(x_i)$, consisting of an embedding matrix, W_E , unembedding matrix, $W_U = W_E^T$, 100 input features and a residual MLP, $MLP_{in} \in \mathbb{R}^{d_{50}, d_{embedding}}$, and similarly $MLP_{out} \in \mathbb{R}^{d_{embedding}, d_{50}}$.

The full computation becomes:

$$y = W_U(W_E(x) + W_{in}(\text{ReLU}(W_E(x))))$$

A naive solution to this problem is to learn a monosemantic decomposition, where each neuron in the MLP is responsible for computing one relu activation. However this will perform poorly on the task, as 50 neurons are forced to guess.

To compare results with the original SPD decomposition, $d_{features} = 100$, $d_{mlp} = 50$, $C = 200$. Predictably, for the MLP_{in} the ground truth mechanism should be to have each subcomponent learn one feature each, however for the MLP_{out} the ground truth mechanism is substantially less clear. Originally presented in APD [Bra+25], they found that the output layer, also learnt a monosemantic representation, where each subcomponent was responsible for one feature. Whereas SPD [BBS25], learns to capture the output layer within 50 subcomponents, where each contributes to the computation of multiple features, and the remaining 150 subcomponents are pruned to near-zero norm.

4.3 Sampling effect on loss

We first take the chance to look at how the sampling distribution affects the decomposition for SPD and Softmax. We compare the one-hot sampling to the randomly initialized one.

Notably we find that Softmax, produces greatly lower losses under one-hot sampling, most certainly because we aim to maximize the probability of ownership, and one-hot sampling removes any noisy or even meaningless features that may come from random activation. sigmoids on the other hand do not have this problem, as they do not enforce competition, but rather if the parts together sum up to be important. We will later see a consequence of this, when exploring the toy model of geometry, where the features become multi-dimensional, and thus require to be grouped together.

Metric	SPD		Softmax	
	One-hot	At-least-zero	One-hot	At-least-zero
Faithfulness				
Stochastic reconstruction				
Layerwise stochastic reconstruction				

Table 4: Reconstruction quality across sampling regimes for the Residual MLP. *To be filled with final numbers.*

4.3.1 Causal importance

In the case of our softmax variant, we also manage to replicate the results of APD, where we manage to find almost one hot decomposition for both the input and output layers, however this interpretation is likely to be misleading.

Plot of causal importances.

Notably, we train the causal importance function to predict the importance of a subcomponent based on $V^T(x)$, the layer input projected into the subcomponents "input" reading.

However, by computing the neuron contribution of each subcomponent, for each feature, we find that the subcomponents manages to produce meaningful contribution to more than one feature each, despite achieving one-hot importance values. Initially this seems paradoxical, as how can a mechanism be completely ablated for one feature, while still contributing meaningful weight to it's output space.

Plot of activation contribution.

This is likely a consequence of the overcomplete basis that the 100 subcomponents learn, namely that if 100 subcomponents learn to project into the same 50 dimensional subspace through a linear mapping, they must necessarily share output direction, and thus contribute to multiple features, even if the importance function assigns them to a single feature.

Plot of cosine similarity of U vectors.

4.3.2 Differences in norm

A second experiment we would like to run is to see, how softmax affects the norms of the subcomponents, likely we hypothesize that a more expressive MLP may learn to disentangle neurons that contribute similarly, and may not even need to shrink the norm to maximize the probability of ownership, as we do not directly enforce the norm to shrink. We show the difference in norms between SPD and Softmax in the figure below. Another hypothesis for this could be that the softmax does not manage to produce "hard zeroes", which was the reason the original SPD paper chose to use

hard sigmoid gating instead. However Plotting the causal importances of the subcomponents, clearly shows that this is not the case, likewise emperical results, with sparsemax produce the same results. [PNM19]

Placeholder plot for causal importance frequency.

Interestingly as expected, all subcomponents produce substantially lower norms, after the first 50 subcomponents. However the norm distribution after this drop off differs substantially between the two methods. In particular, SPD manages to prune the norm to zero for all remaining subcomponents, where as softmax has many subcomponents with low norm, without being causally important for any features.

Referencing back to the TMS experiment, this is likely part of the explanation for why we observed significant feature shrinkage. The faithfulness loss, is trained to minimize the MSE between the layers weight matrix, while it does not care where it is distributed, as long as they sum up to be equal. That means if the minimality loss only partially ablates the weights to be "insignificant", albeit non-zero, the faithfulness loss will not reassign the missing norm to the dominant subcomponents.

4.3.3 Cause of the lying?

Returning to the question of one-hot importance, we fear that this may be an illusion of interpretability, or while not an illusion it is misleading. The model has very likely learnt to disambiguate the subcomponents, but the subcomponents or perhaps particularly their U matrices, are not constrained to project orthogonally to eachother and are not represented in this importance calculation.

Also need to relate back to the multidimensional feature hypothesis.

4.3.4 Sampling regimes

We compare two sampling regimes for the masks during stochastic reconstruction: **one-hot sampling**, where exactly one subcomponent is forced active per feature, and **at-least-zero sampling**, the relaxed regime used during normal training. Both regimes converge to similar reconstruction quality, but they place different pressures on the structure of the recovered decomposition.

Metric	SPD		Softmax	
	One-hot	At-least-zero	One-hot	At-least-zero
Faithfulness				
Stochastic reconstruction				
Layerwise stochastic reconstruction				

Table 5: Reconstruction quality across sampling regimes for the Residual MLP. *To be filled with final numbers.*

4.3.5 Causal importance

The softmax variant achieves near-perfect one-hot causal importance, with each feature routing essentially all its mass to a single subcomponent (Figure 4). The histogram of importance values is bimodal at 0 and 1, with negligible off-peak mass — the same property the original SPD method achieves through its sigmoid-and-minimality combination, here recovered through softmax-with-entropy.

Figure 4: Causal importance values for the softmax variant on the Residual MLP. *Left*: heatmaps for W_{in} and W_{out} , showing near-diagonal one-hot routing of features to subcomponents. *Right*: histogram of all importance values, bimodal at 0 and 1.

This is a non-trivial result on its own. Replacing the importance minimality loss with entropy preserves the routing property that SPD’s heatmaps are known for, suggesting that the routing structure can be recovered without the Schatten-style parameter penalty that the original method relies on.

4.3.6 Neuron contributions

Following [Bra+25], we visualize the per-neuron contribution of each subcomponent compared to the target model’s per-feature neuron contributions. For each feature, we identify the subcomponent with the largest neuron contribution (the “dominant” subcomponent) and plot its full neuron pattern alongside the target’s.

Figure 5: Per-neuron contributions for the Residual MLP. *Top*: target model contributions for the first 30 features, with the top-contributing neurons labelled. *Bottom*: the dominant SPD subcomponent for each feature, with neuron labels propagated from the target panel.

The dominant subcomponents recover the correct neurons for each feature, matching the target’s labels. However, the magnitudes are visibly shifted: the subcomponent contributions do not consistently reproduce the target’s magnitudes, with the same neuron often appearing across multiple subcomponents at non-trivial weight. Despite clean routing, the parameter directions are not isolated to single features.

This is a different failure mode than TMS exhibits. In TMS the softmax variant produced shrinkage on the dominant subcomponents themselves; here, the dominant subcomponents are approximately correct, but additional subcomponents carry residual weight on the same neurons.

4.3.7 Subcomponent norms

To make this concrete, we plot the Frobenius norm $\|A_c B_c\|_F$ of each rank-1 subcomponent, sorted by magnitude (Figure 6).

Figure 6: Sorted Frobenius norms $\|A_c B_c\|_F$ of all $C = 200$ subcomponents for W_{in} (left) and W_{out} (right). A clear cliff separates roughly 80 high-norm subcomponents from a long tail of ~ 120 low-but-nonzero subcomponents.

The distribution is bimodal but not clean. The first ~ 80 subcomponents sit at full norm (~ 1.0 – 1.4 for W_{in} , ~ 2.0 for W_{out}), followed by a sharp drop, and then a long tail of ~ 120 subcomponents at norm 0.15 – 0.35 . The tail is small relative to the head but unambiguously **non-zero** — these subcomponents have not been pruned.

This contrasts with the TMS_{5.2} result, where the softmax variant correctly pruned the unused subcomponents to near-zero norm despite the ML2R shrinkage on the active ones. In the Residual MLP setting, neither property is fully recovered: active subcomponents experience some magnitude drift, and inactive subcomponents are not driven to zero.

4.3.8 Sensitivity to hyperparameters

4.4 Softmax without attention

4.4.1 Noes for this

To quantify this, the effective rank of each pair’s submatrix, $W_{:,2i-1:2i} \in \mathbb{R}^{d_{\text{hidden}} \times 2}$ is computed using the entropy-based definition of [RV07].

$$r_{\text{eff}}(W_g) = \exp\left(-\sum_{k=1}^Q p_k \log p_k\right), \quad p_k = \frac{\sigma_k^2}{\sum_{j=1}^Q \sigma_j^2} \quad (18)$$

where σ_k are the singular values of W_g and $Q = |g|$. The quantity lies in $[1, |g|]$ and equals 1 when the group is exactly rank-one. To gauge the required value of k for the rank- k extension, the maximum effective rank across pairs is examined:

$$k = \max_i r_{\text{eff}}(W_{:,2i-1:2i}), \quad (19)$$

In this case the max is essentially 1 across all pairs, suggesting that $k = 1$ is sufficient to capture these features without loss of information. (Table 6).

Mean	Std Dev	Range
1.0003	0.01	[1.0, 1.05]

Table 6: Effective rank of paired-feature encoder blocks. Values near 1 indicate collapse to a single direction.

Remarks for this section Intra cluster cosine has a -1. Two features are perfectly aligned to a factor of -1, can this cause issues with decomposition? Same component could capture both features, if they read from the same input. However they shouldnt as its distinctly different feature indices.

4.5 Extending to Rank-k: When Higher-Dimensional Structure Matters

The rank-k extension is required in settings where feature representations do not collapse to a single direction. This is illustrated by a model trained to compute the volume of an n-dimensional simplex.

In this experiment, consider a model trained to compute the volume of an n-dimensional simplex:

$$f(x) = W_1 \sigma(W_2 x), \quad W_2 \in \mathbb{R}^{k \cdot n(n+1) \times d_{\text{hidden}}}, \quad W_1 \in \mathbb{R}^{d_{\text{hidden}} \times k} \quad (20)$$

Here, k denotes the number of simplices and n the dimensionality of each simplex, which has n+1 vertices.

As in Section 4.2.2, the k simplices are sampled independently with fixed probability, such that either all n(n+1) features of a simplex are active or none are. The resulting feature groups are therefore statistically dependent, and their joint distribution assigns negligible probability to partial activation patterns. By the definition of irreducibility in Section 3.2, each simplex constitutes an irreducible feature with formal dimensionality $d_{f_i} = n(n+1)$.

For this experiment, the model is trained with $n = 2$ and $k = 4$, yielding four triangles $\mathbb{R}^2$ and a total of 24 input features. The ground-truth decomposition therefore consists of four subcomponents, each responding to 6 adjacent features. We note that the effective rank and cosine similarity here imply a decomposition should require at least $k = 2$. **[Placeholder: cosine similarity results]**

[Placeholder: effective rank results, showing $r_{\text{eff}} > 1$] Firstly, we demonstrate that the original SPD formulation surprisingly yields the best clustered reconstruction when $k = 1$:

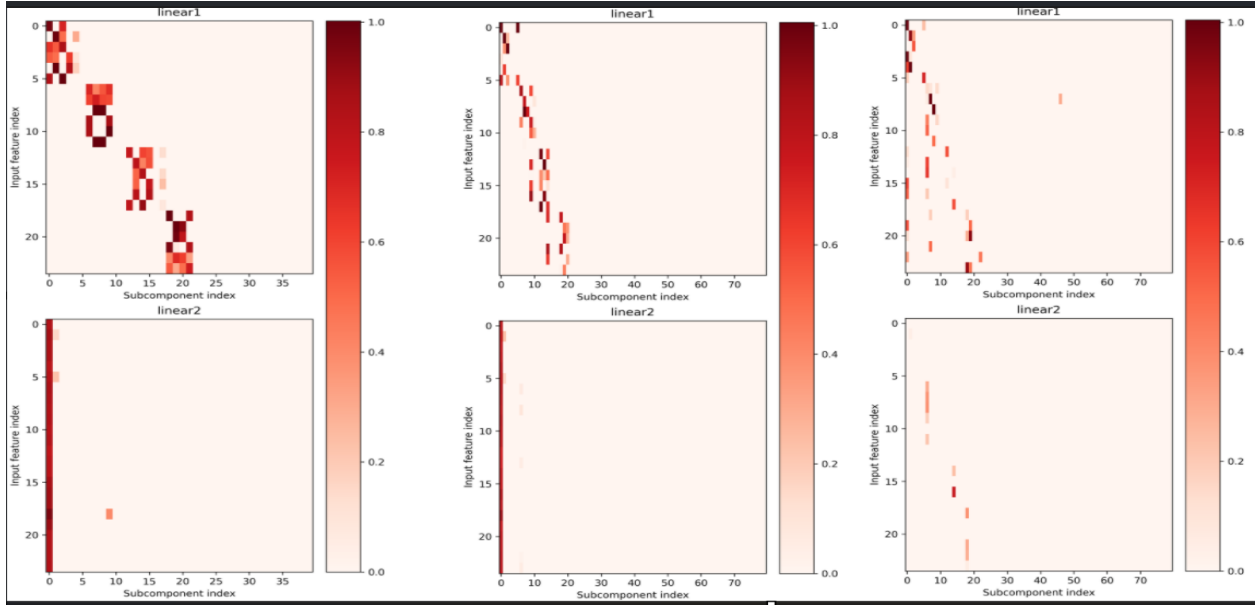


Figure 7: SPD decomposed with k=1,2,3 from left to right.

The reconstruction clearly yields 4 subcomponents per group, that only respond to the 6 features of a single simplex, however these subcomponents are separated even though they are irreducible.

We note that we weren't able to find a reconstruction at all when the minimality coefficient was outside of $[1e-4, 5e-4]$.

The softmax and attention extension, on the other hand, shows the expected behaviour, with $k = 1$, the reconstruction necessitates that each simplex is split into multiple subcomponents, here each subcomponent captures a single feature. The extension to $k = 2$ allows the model to capture each simplex as a single unit, with all 6 features of a simplex captured by a single subcomponent. However, it does collapse 2 simplices into a single subcomponent. The extension to $k = 3$ does not yield any improvement, suggesting that the model has found a faithful reconstruction at $k = 2$.

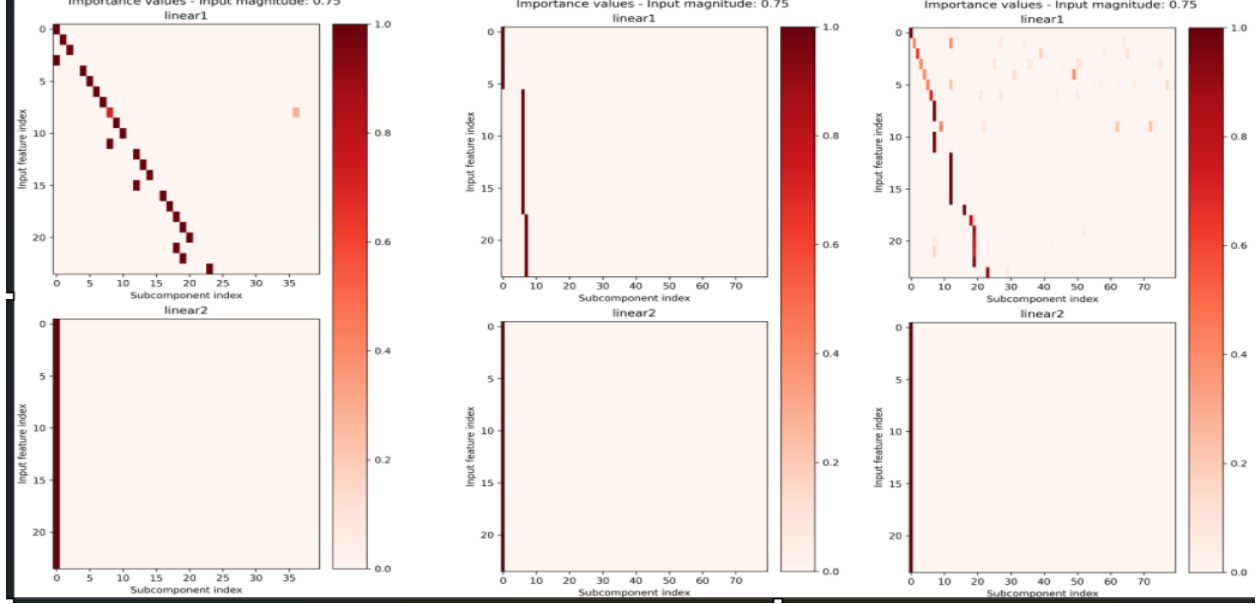


Figure 8: Softmax-and-Attention decomposed with $k=1,2,3$ from left to right.

While the softmax approach does not find a perfect reconstruction either, it clearly demonstrates how the extension from rank 1 to rank 2 allows a non diagonal representation of the features. Had it been perfect, then the middle plot would have shown 4 components instead of the collapsed 2.

Same plots i was going to do for the TMS experiment, perhaps this gets its own section, we compare how input spaces are alligned over training, does the softmax encourage δ orthogonality better?

4.5.1 things that didnt make it into tms paired

To verify a successful decomposition, each matrix V_i should also be δ -orthogonal to the others.

The multi-dimensional superposition hypothesis predicts that the projection matrices for different irreducible features are pairwise δ -orthogonal. At the parameter level, the recovered V_c matrices are the operational counterpart, and the prediction translates to the requirement that recovered atoms have small pairwise overlap in their input-side directions.

For each method, we compute the pairwise cosine similarity⁹

$$M_{cc'} = \frac{V_c^\top V_{c'}}{\|V_c\| \|V_{c'}\|} \quad (21)$$

compared against the across-group cosine similarity of the trained model's encoder (0.053, 2), the softmax-and-attention shows "something" and the baseline model finds that "something else".

here we have four nice plots, overall alligment, across V's, across U's, entire thing? Reference to original model and maybe a table?

4.6 Checking for orthogonality throughout training with softmax and attention / compare to baseline spd.

4.7 New toy model

Quantization paper computes the parity of a string mod 2, it proposes control bit sequence, that decide which subset of the string that is active and should be computed on,

we can sample this with the mixture

$$\text{mixture} = p(\text{string}, \delta_{\text{control}=0}) + p(\text{string}, \delta_{\text{control} \neq 0})$$

⁹Maybe even UV combined? Only read or write?

4.8 Toy model of learning gates

Remember to mention that softmax is all or nothing, we have to one hot sample here afterwards! However we need to do this on an affine transformation no?

We learn $(a \wedge b) \vee a$. Ground truth interestingly becomes one component active for each similar directions but project into the same output direction.

Remark: Im not sure we can sample binary values, double check the distribution, i think it requires a continuous thing with b not fixed? Could always train an addition model, but they already did that in the original paper? Can we just use that one to prove the claim?

To test the case of enges, where the probability distribution is in fact reducible. We could train it to learn logic gates i.e $1+0=0$, $1+1=1$, $0+0=0$, $0+1=1$, in such a view each could occur independently ,but nlywhen both are active, the output is active, we then sample independently from the two features and see if it compacts them in a single unit or not.

The concept here is clear: The ideal decomposition here is one that produces independent features, that can co occur, however the atomic solution is independence.

We also need to probe for a paired result here. The ground truth should show part ownership across 2 components. bool A, bool B are independnet their joint result shoul be shared ownership.

4.8.1 Remark for experiments

Consider trainng on exactly one active.

5 Discussion

The original spd method produces hard zeroes, something that softmax cannot do. Entmax didnt work either nor did sparsemax, however non hard zeroes do cause magnitude to leak even if marginal, which may increase with more components.. important to mention that the str ucture that spd decomposes into is not clearsometimes one hots ome.

Talk about how its unclear what structure spd wants. Even rank 1 is not the smallest possible, as we can align two things to the same direction, but these do not split.

Relate back to the interesting thing that one feature per component and rank 1 matrices does in fact not mean the smallest possible decomposition, as we have shown with TMSs and TMS paired, one is in a way strictly larger, but they result in the same thing!

6 Acknowledgements

7 Other things to consider?

Minimal descriptive length?

7.1 That some features are multidimensional

Think back to SAE, that learn a dictionary of features, then find hte linear combination of those features, however that this fails for cyclical features, as its cheaper to learn a single feature, than learn a direction and a periodic function. Now for your experiment, the dream scenario is that we allowe a multi dimensional component, and that this can learn the cyclical feature, i.e a weekly cycle or modular arithmetic either do this or extend TMS to be multi-dimensional, show if it works better with ur gnn or not, likewise we need to show that through individual masking of features, we can teach a component to kill one of its features, if it is better explained by a rank 1 component than rank 2.

8 References

[BBS25] Lucius Bushnaq, Dan Braun, and Lee Sharkey. *Stochastic Parameter Decomposition*. 2025. arXiv: 2506.20790 [cs.LG]. URL: <https://arxiv.org/abs/2506.20790>.

- [Bha+25] Jai Bhagat et al. *Compressed Computation is (probably) not Computation in Superposition*. <https://www.lesswrong.com/posts/ZxFchCFJFcgySyt9/compressed-computation-is-probably-not-computation-in>. LessWrong post. June 2025.
- [Bra+25] Dan Braun et al. *Interpretability in Parameter Space: Minimizing Mechanistic Description Length with Attribution-based Parameter Decomposition*. 2025. arXiv: 2501.14926 [cs.LG]. URL: <https://arxiv.org/abs/2501.14926>.
- [Bri+23] Trenton Bricken et al. “Towards Monosemanticity: Decomposing Language Models With Dictionary Learning”. In: *Transformer Circuits Thread* (2023). <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- [CBS25] Brianna Chrisman, Lucius Bushnaq, and Lee Sharkey. *Identifying Sparsely Active Circuits Through Local Loss Landscape Decomposition*. 2025. arXiv: 2504.00194 [cs.LG]. URL: <https://arxiv.org/abs/2504.00194>.
- [Cha+25] David Chanin et al. *A is for Absorption: Studying Feature Splitting and Absorption in Sparse Autoencoders*. 2025. arXiv: 2409.14507 [cs.CL]. URL: <https://arxiv.org/abs/2409.14507>.
- [CR25] Casper L. Christensen and Logan Riggs. *Decomposition of Small Transformer Models*. 2025. arXiv: 2511.08854 [cs.LG]. URL: <https://arxiv.org/abs/2511.08854>.
- [Cso+24] Róbert Csordás et al. *Recurrent Neural Networks Learn to Store and Generate Sequences using Non-Linear Representations*. 2024. arXiv: 2408.10920 [cs.LG]. URL: <https://arxiv.org/abs/2408.10920>.
- [Cun+23] Hoagy Cunningham et al. *Sparse Autoencoders Find Highly Interpretable Features in Language Models*. 2023. arXiv: 2309.08600 [cs.LG]. URL: <https://arxiv.org/abs/2309.08600>.
- [DCN24] Jacob Dunefsky, Philippe Chlenski, and Neel Nanda. *Transcoders Find Interpretable LLM Feature Circuits*. 2024. arXiv: 2406.11944 [cs.LG]. URL: <https://arxiv.org/abs/2406.11944>.
- [Elh+22] Nelson Elhage et al. *Toy Models of Superposition*. 2022. arXiv: 2209.10652 [cs.LG]. URL: <https://arxiv.org/abs/2209.10652>.
- [Eng+25] Joshua Engels et al. *Not All Language Model Features Are One-Dimensionally Linear*. 2025. arXiv: 2405.14860 [cs.LG]. URL: <https://arxiv.org/abs/2405.14860>.
- [JT24] Adam Jermy and Adly Templeton. *Tanh Penalty in Dictionary Learning*. Section in the February 2024 update post. Transformer Circuits. 2024. URL: <https://transformer-circuits.pub/2024/feb-update/index.html> (visited on 05/10/2026).
- [Lea+25] Patrick Leask et al. *Sparse Autoencoders Do Not Find Canonical Units of Analysis*. 2025. arXiv: 2502.04878 [cs.LG]. URL: <https://arxiv.org/abs/2502.04878>.
- [Mar+25] Samuel Marks et al. *Sparse Feature Circuits: Discovering and Editing Interpretable Causal Graphs in Language Models*. 2025. arXiv: 2403.19647 [cs.LG]. URL: <https://arxiv.org/abs/2403.19647>.
- [Mél+25] Maxime Méloux et al. *Everything, Everywhere, All at Once: Is Mechanistic Interpretability Identifiable?* 2025. arXiv: 2502.20914 [cs.LG]. URL: <https://arxiv.org/abs/2502.20914>.
- [Men24] Jake Mendel. *SAE Feature Geometry is Outside the Superposition Hypothesis*. AI Alignment Forum. June 2024. URL: <https://www.alignmentforum.org/posts/MFBTjb2qf3ziWmzz6/sae-feature-geometry-is-outside-the-superposition-hypothesis>.
- [Nan+23] Neel Nanda et al. *Progress measures for grokking via mechanistic interpretability*. 2023. arXiv: 2301.05217 [cs.LG]. URL: <https://arxiv.org/abs/2301.05217>.
- [Ola+20] Chris Olah et al. “Zoom In: An Introduction to Circuits”. In: *Distill* 5.3 (2020). DOI: 10.23915/distill.00024.001. URL: <https://distill.pub/2020/circuits/zoom-in/>.
- [PCV24] Kiho Park, Yo Joong Choe, and Victor Veitch. *The Linear Representation Hypothesis and the Geometry of Large Language Models*. 2024. arXiv: 2311.03658 [cs.CL]. URL: <https://arxiv.org/abs/2311.03658>.
- [PNM19] Ben Peters, Vlad Niculae, and André F. T. Martins. *Sparse Sequence-to-Sequence Models*. 2019. arXiv: 1905.05702 [cs.CL]. URL: <https://arxiv.org/abs/1905.05702>.
- [RV07] Olivier Roy and Martin Vetterli. “The effective rank: A measure of effective dimensionality”. In: *2007 15th European Signal Processing Conference*. 2007, pp. 606–610.
- [Sha+25] Lee Sharkey et al. *Open Problems in Mechanistic Interpretability*. 2025. arXiv: 2501.16496 [cs.LG]. URL: <https://arxiv.org/abs/2501.16496>.
- [Smi24] Lewis Smith. *The ‘strong’ feature hypothesis could be wrong*. LessWrong. Accessed: 2026-05-03. Aug. 2024. URL: <https://www.lesswrong.com/posts/tojtPCCRpKLSHBdpn/the-strong-feature-hypothesis-could-be-wrong>.

- [Vas+23] Ashish Vaswani et al. *Attention Is All You Need*. 2023. arXiv: 1706.03762 [cs.CL]. URL: <https://arxiv.org/abs/1706.03762>.
- [WS24] Benjamin Wright and Lee Sharkey. *Addressing Feature Suppression in SAEs*. AI Alignment Forum. Feb. 16, 2024. URL: <https://www.alignmentforum.org/posts/3JuSjTZyMzaSeTxKk/addressing-feature-suppression-in-saes> (visited on 05/10/2026).

9 Appendix

Descriptive length thing? + approximation of attention experiment

9.0.1 GNAN-based importance functions

In SPD, each importance function γ_c^l is an independent MLP that receives only its own inner activation. Intuitively, however, causal importance is relational: whether a subcomponent can be ablated without affecting the output may well depend on which other subcomponents are active alongside it.

To address this, the independent MLPs are replaced with a Graph Neural Additive Network (GNAN) that can relate each subcomponent to its neighbourhood.

The C subcomponents are treated as nodes in a graph, with node features given by their inner activation vectors $\mathbf{h}_c^l(x) \in \mathbb{R}^k$. The GNAN updates each node’s representation through an additive neighbourhood aggregation. For node i in layer l and feature dimension k ¹⁰:

$$[\mathbf{z}_i]_k = \sum_{j=1}^N \frac{1}{|\mathcal{L}(j)|} \cdot \rho\left(\frac{1}{1 + \text{dist}(j, i)}, \cos(j, i)\right) \cdot f_k([\mathbf{h}_j]_k) \quad (22)$$

OBS: Implementation ill present today is this:

$$\mathbf{z}_i = \sum_{j=1}^N \rho(\text{layer distance, cosine similarity}) \cdot f_k([\mathbf{h}_j]_k) \quad (23)$$

where f_k is a learned function applied to the k -th inner activation of each neighbouring node j ; $|\mathcal{L}(j)|$ is the number of nodes in the same layer as j , which normalises the contribution of each layer; $\text{dist}(j, i)$ is the layer distance between nodes j and i ¹¹; $\cos(j, i)$ is the cosine similarity between their parameter vectors; and ρ is a learned function that maps both the distance kernel and the cosine similarity to a scalar weight.

$\mathbf{z}_i$ becomes a vector in $\mathbb{R}^k$, which fits perfectly with the current activation vectors.

A separate MLP is trained to combine the aggregated neighbourhood information and the node’s own activation into an importance value in $[0, 1]$: Producing a single importance value per subcomponent and passing it through a leaky hard sigmoid, following SPD [BBS25]:

$$g_c^l = \sigma_H(\text{AGG}(\mathbf{z}_i^l, \mathbf{h}_i^l)) \quad (24)$$

The self-connection through f_{self} (or just the raw activation) ensures that each node retains its own identity after aggregation, mitigating the oversmoothing that can occur in GNNs. {Clever cite here, i.e gnn book}

Alternative formulation, that would work with the next section

After the GNAN update, the per-dimension importance values are computed from the updated representation and passed through a leaky hard sigmoid, following SPD [BBS25]:

$$\mathbf{g}_c^l(x) = \sigma_H(\mathbf{h}_c^l(x)) \in [0, 1]^k \quad (25)$$

in other words, the importance function becomes multidimensional, and each column can be masked independently.

Remarks:

For now the edge set construction is left as the fully connected graph, which remains reasonably computationally expensive for small models. However it could very easily be made more efficient, for other architectures, or by

¹⁰I am very open to alternative equations, i.e perhaps we delete the layer distance inside rho, however it seemed necessary to use cosine similarity, as otherwise the signal was useless as described in some earlier section

¹¹The layer distance, could later be signed to differentiate between previous and future layers

considering only the k-hop neighbourhood of each node, or only considering prior neighbours or later neighbourhoods. Likewise we could store both distances as sparse structures as they are symmetric. $\cos(j, i) = \cos(i, j)$

Remark 2:

The choice of edge set construction and distance function is a very important design choice, e.g using the layer distance on a fully connected graph encodes the same signal for all nodes in a layer: For nodes i_1, i_2 in the same layer l :

$$\forall j: \quad \text{dist}(j, i_1) = \text{dist}(j, i_2) = |k_j - l| \quad (26)$$

and therefore

$$[\mathbf{h}_{i_1}]_k = \sum_{j=1}^N \frac{1}{\#\text{dist}_{i_1}(j, i_1)} \cdot \rho\left(\frac{1}{1 + |k_j - l|}\right) \cdot f_k([\mathbf{x}_j]_k) = [\mathbf{h}_{i_2}]_k \quad (27)$$

since no term in the sum depends on the identity of i within layer l , only on the layer index l itself.

Additionally this produces a counterproductive signal, the gnan tries to boost or suppress all nodes in a layer together, instead of learning to differentiate between them.

Remark 3:

The choice of distance should probably be reconsidered in general, I have a feeling that the function needs to be from an asymmetric metric space, i.e

$$\text{dist}(i, j) \neq \text{dist}(j, i) \quad (28)$$

Atleast if the intuition is if component A in layer k, is highly active, perhaps component B in layer k+1 should likely be inactive, as the other component needs to handle this feature, thus the distance from A to B should be different, as otherwise both are boosted or suppressed together.

Remark 4:

The current experiments are more sensitive to hyperparameters, than the original code seems to be, if the minimality pressure is too low, then all components are active, and doesn't "kill" any of them

Remark 5:

I haven't been able to find any "benefit" in setting k higher, this is ofcourse because the ground truth of all the models we have are rank 1 independent features, we need to find a scenario where the dependence and or rank k is actually necessary:

My ideas

- Sample points from the unit circle, i.e $\cos(\theta)^2 + \sin(\theta)^2 = 1$ then

$$x = \sqrt{1 - \sin(\theta)^2} \text{ or } -\sqrt{1 - \sin(\theta)^2}$$

i.e something can only be expressed with a rank 2 component?

- we could look for circuits, the goal is then no longer to find N independent features, but suppose we have input groups [0,1,2], [2,3,4], [3,4,5] then we want one component that activates for the first group, one for the second and one for the third

Remark 6:

I added the gnan plots to wandB remember to show them

Remark 7:

The per feature construction with only 1 feature is rather boring, and does not utilize the expressiveness at all.

Remark 8:

I dont think its a code problem, as i can reproduce equivalent results to their findings. Im just not able to beat them. We need to discuss experiments where the k dimensionality and dependence actually matters.

10 Notes

Putting the minimality loss entropy higher, encourages diagonals? Instead of capturing two features in the same component.

Increasing minimality loss encourages feature grouping.

SAE: learn to decompose activations, i.e the output of a layer, Transcoders: learn to approximate the input-output function of a layer

$$\int p(a, b) da db = w \int p_A(a) da \int \delta(b) db + (1 - w) \iint p_{AB}(a, b) da db = w + (1 - w) = 1$$

to read: Terence Tao, *An Introduction to Measure Theory*: Section 1.1 Prologue: The problem of measure — sets up why measure theory exists and why "lower-dimensional sets have measure zero" is the central idea. This alone might give you the intuition you need. Section 1.2 Lebesgue measure — gives you "measure zero," "Lebesgue measure on $\mathbb{R}^n$," and the fact that lines/planes/etc. have measure zero in higher-dimensional ambient spaces. This is the technical core for your question. Section 2.3 Probability spaces — short section that frames everything in probability language, which is what you actually want.

10.0.1 Hard maybes

10.0.2 Per-dimension ablation and implicit rank selection (Hypothesis: ask lukas)

Following the extension to rank- k subcomponents, and the alternative formulation of the GNAN, the output $g_c^l \in [0, 1]^k$ is a per feature importance value, and thus the masking can be applied independently to each feature. The intuition behind this is that if a mechanism captures a 1 dimensional feature, it would only need of the k dimensions to be active.

Likewise if this actually ends up working, k can be considered an upper bound or a so called "capacity" of a mechanism, it also follows nicely with the descriptive length principle found in the appendix of APD.

To determine the effective rank of each subcomponent, the stochastic masking becomes:

$$[m_c^l(x, r)]_j = [g_c^l(x)]_j + (1 - [g_c^l(x)]_j) r_{c,j}^l, \quad r_{c,j}^l \sim U(0, 1) \quad (29)$$

for each j in $1, \dots, k$

Notably this treats each feature independently, which may not be desirable, as such we could modify it ever so slightly:

$$[m_c^l(x, r)]_j = [g_c^l(x)][g_c^l(x)]_j + (1 - [g_c^l(x)][g_c^l(x)]_j) r_{c,j}^l, \quad r_{c,j}^l \sim U(0, 1) \quad (30)$$

The signal becomes an importance of two factors, if the component is important and both features are important, it remains active, if only one feature is important, the other will be pushed to zero

The masked weight matrix becomes:

$$W^l(x, r) = \sum_{c=1}^C U_c^l \text{diag}(\mathbf{m}_c^l(x, r)) V_c^{l\top} \quad (31)$$

Where the diagonal matrix is the per-feature mask for subcomponent c .

The importance minimality loss penalises all k dimensions of all subcomponents toward zero:

$$\mathcal{L}_{\text{import}} = \sum_{l=1}^L \sum_{c=1}^C \sum_{j=1}^k |[g_c^l(x)]_j|^p \quad (32)$$

This follows the same logic by which SPD selects the number of active subcomponents: the minimality loss drives unused components toward zero, and only those that are genuinely required to reconstruct the target model's output survive.

This makes k a soft upper bound rather than a sensitive hyperparameter. In practice, k can be set conservatively large and the effective rank of each subcomponent is learned from the data.

11 To add:

<https://arxiv.org/pdf/2303.13506> quantization, multidimensional feature crossover We use the method from the irreducibility paper to quantize how well it is seperable, mixture like. Rewrite irreducibility to mkae it clear that it is the function

The method is quite complex, we practically look for the input space to see where a feature comes from and its dimension but when we look to decompose a layer, we look at f , the projection of the input space to see its rank, but we look for the values that the input space get mapped to. So in other words, the f decides how many subcomponents we need, but the f decides how many features are inside of it?

quantization and spd clustering

why softmax + attention instead of only softmax + minimality.

Somewhere add this whole thing of importance maybe being an interpretability trap? Or that it should be seen as a caution thing, it will predict what is responsible, not the model alignment.

The thing that makes multifeature hypothesis perfect

Multifeature hypothesis says that there exists a transformation f , to a lower dimensional space, in this case any W linear map,

that can be paired with some new V_i that describes the hidden state, in this case it's the SPD components!!!! These perfectly make this up and pair this with quantization and we are golden.

Remark rewrite TMS paired into TMS, they should be the same section. Remember V is the input reading so if w